

Section 10: Detection Engineering

20 detections total (10.1–10.17 endpoint/network, 10.18–10.20 perimeter/Sophos — added in the integration pass; earlier drafts of this file mislabeled the count as "18" while only enumerating 17, see project audit log).

Format per detection: Description → MITRE ATT&CK → Sigma Rule → Detection Logic → Severity → False Positives → Investigation Steps → Recommended Response.

Sigma rules here target `logsource: product: windows` (Sysmon/Security), `product: linux` (auditd/sshd), or a custom `sophos-firewall` Wazuh decoded source (10.18–10.20 — Sigma's generic schema doesn't have a native Sophos backend, so those three are given as Wazuh-native rule XML directly instead of Sigma YAML — noted per-rule). Convert Sigma rules to Wazuh-native with `sigma-cli` (`pip install sigma-cli`, then `sigma convert -t opensearch <rule.yml>`) or hand-port into `/var/ossec/etc/rules/local_rules.xml`.

10.1 PowerShell Execution (baseline visibility)

Description: Detects any PowerShell process spawn — the baseline rule that everything else in this category refines. **MITRE ATT&CK:** T1059.001 (Command and Scripting Interpreter: PowerShell)

title: PowerShell Process Execution

logsource: {product: windows, category: process_creation}

detection:

selection:

Image|endswith: '\powershell.exe'

condition: selection

level: low

Detection Logic: Sysmon Event ID 1 where `Image` ends in `powershell.exe` or `pwsh.exe`.

Severity: Low (informational baseline — too broad alone to act on). **False Positives:** Extremely high — legitimate admin scripts, scheduled maintenance, software installers all use PowerShell.

Investigation Steps: Pull `CommandLine`, `ParentImage`, `User` from the Sysmon event. Legitimate PowerShell from `explorer.exe` by a known admin ≠ PowerShell spawned from `winword.exe` or `outlook.exe` (macro-drop pattern). **Recommended Response:** No action on this rule alone — it exists to feed the more specific rules below. Tune it out of the primary alert queue; keep it in raw logs for hunting.

10.2 Encoded PowerShell (obfuscation)

Description: Detects Base64-encoded PowerShell (`-EncodedCommand` / `-enc`), a standard obfuscation and AV-evasion technique. **MITRE ATT&CK:** T1027 (Obfuscated Files or Information), T1059.001

title: Encoded PowerShell Command

logsource: {product: windows, category: process_creation}

detection:

selection:

Image|endswith: '\powershell.exe'

CommandLine|contains:

- '-enc'

- '-EncodedCommand'

- '-e '

condition: selection

level: high

Detection Logic: Sysmon ID 1, `CommandLine` contains an encoding flag. **Severity:** High. **False Positives:** Low-moderate — some legitimate deployment tools (SCCM, Intune scripts) use encoded commands to avoid quoting issues; whitelist by `ParentImage` for known management agents. **Investigation Steps:** Decode the Base64 blob (CyberChef: `From Base64 → Decode text (UTF-16LE)` — PowerShell encodes as UTF-16LE) to recover the actual command. Check `ParentImage` for office apps or browsers (drive-by/macro indicator).

Recommended Response: Isolate host if decoded payload contains download cradles ([IWR](#), [Net.WebClient](#)) or reflective loading patterns. Escalate to Tier 2.

10.3 Failed Logins

Description: Individual authentication failure, Windows or Linux. **MITRE ATT&CK:** T1110 (Brute Force) — precursor telemetry

title: Windows Failed Logon

logsource: {product: windows, service: security}

detection:

selection:

EventID: 4625

condition: selection

level: informational

Detection Logic: Windows Security Event ID 4625, or Linux [sshd](#) "Failed password" in [/var/log/auth.log](#) (Wazuh rule ID 5503/5710 range). **Severity:** Informational (single event); escalates when aggregated (see 10.5). **False Positives:** Users mistyping passwords — normal background noise. **Investigation Steps:** Check [TargetUserName](#), [IpAddress](#), [LogonType](#). A single failure from a known user's known IP = ignore. Failures against [Administrator/root](#) or non-existent accounts = suspicious. **Recommended Response:** Log only, unless part of a threshold pattern (10.5).

10.4 Successful Logins (anomalous)

Description: Not "any successful login" (too noisy) — specifically successful logons that are anomalous in time, location, or method. **MITRE ATT&CK:** T1078 (Valid Accounts)

title: Anomalous Successful Logon

logsource: {product: windows, service: security}

detection:

selection:

EventID: 4624

LogonType: [3, 10] # 3=network, 10=RDP

filter_known_hours:

hour|range: '9-18'

condition: selection and not filter_known_hours

level: medium

Detection Logic: Event ID 4624 outside business hours, or **LogonType** 10 (RemoteInteractive) from an unexpected source IP. **Severity:** Medium. **False Positives:** Legitimate off-hours admin work, different time zones — needs a maintained allowlist of expected admin behavior.

Investigation Steps: Correlate **IpAddress**, **LogonType**, **TargetUserName** against asset inventory / expected admin list. **Recommended Response:** Verify with account owner out-of-band before escalating to containment.

10.5 Brute Force

Description: Threshold-based detection — N failed logins against the same account/source within a short window. **MITRE ATT&CK:** T1110.001 (Password Guessing)

title: SSH Brute Force Attempt

logsource: {product: linux, service: sshd}

detection:

selection:

message|contains: 'Failed password'

timeframe: 5m

condition: selection | count(by=src_ip) > 10

level: high

Detection Logic: Wazuh native `frequency/timeframe` rule — 10+ auth failures from one source IP within 5 minutes (mirrors the built-in `active-response` brute-force ruleset, IDs 5712-5716 for SSH, 60204+ for Windows RDP). **Severity:** High. **False Positives:**

Misconfigured automated jobs retrying with stale credentials, vulnerability scanners.

Investigation Steps: Confirm source IP externality, check if any attempt succeeded (correlate with 10.4). Check MISP for the source IP (Section 4 of the main pipeline doc). **Recommended**

Response: Auto-block via Wazuh **active response** (`firewall-drop` command) after threshold; manually verify before permanent block if source could be internal.

10.6 Port Scan

Description: Detects horizontal/vertical port scanning against a host. **MITRE ATT&CK:** T1046 (Network Service Discovery) **Detection Logic:** Suricata rule (not Sysmon-visible) — connections to >15 distinct destination ports from one source within 60 seconds:

```
alert tcp any any -> $HOME_NET any (msg:"Possible Port Scan Detected"; \
```

```
flags:S; threshold:type both, track by_src, count 15, seconds 60; \
```

```
classtype:attempted-recon; sid:1000001; rev:1;)
```

Severity: Medium. **False Positives:** Legitimate network/asset discovery scans by your own vulnerability management tooling — whitelist that IP. **Investigation Steps:** Identify scan type (SYN/full-connect) via Wireshark capture, check which ports responded (indicates what the attacker learned). **Recommended Response:** No immediate containment for scan alone (recon ≠ compromise) — but tag source IP for heightened monitoring on subsequent alerts.

10.7 Privilege Escalation

Description: Detects a process gaining SYSTEM/root context it shouldn't have, or `sudo/UAC` bypass patterns. **MITRE ATT&CK:** T1548 (Abuse Elevation Control Mechanism)

title: Suspicious Privilege Escalation via Token Manipulation

logsource: {product: windows, category: process_creation}

detection:

selection:

CommandLine|contains:

- 'whoami /priv'
- 'runas /user:'

ParentImage|endswith: '\cmd.exe'

condition: selection

level: high

Linux equivalent — auditd rule on **execve** for **sudo/su** outside normal admin accounts, keyed on **identity_change** (Section 9.1). **Severity:** High. **False Positives:** Legitimate admin troubleshooting. **Investigation Steps:** Check the full process tree — was this preceded by a suspicious parent (e.g., a script or downloaded binary), or a known admin's interactive session? **Recommended Response:** If preceded by non-interactive parent (script/service) → treat as compromise, isolate host.

10.8 New User Creation

Description: Detects account creation, a common persistence step post-compromise. **MITRE ATT&CK:** T1136 (Create Account)

title: New User Account Created

logsource: {product: windows, service: security}

detection:

selection:

EventID: 4720

condition: selection

level: medium

Detection Logic: Windows Event ID 4720; Linux — auditd `identity_change` on `/etc/passwd` write, or `useradd/adduser` execve. **Severity:** Medium (High if account is added to privileged groups shortly after — chain with 4732). **False Positives:** Legitimate IT onboarding — correlate against change-management ticket if available. **Investigation Steps:** Check who created the account (`SubjectUserName`), what group it was added to next, whether it was created outside business hours. **Recommended Response:** Disable account pending verification if creator/context is unexplained.

10.9 Persistence — Scheduled Tasks

Description: Detects new scheduled task creation, a top persistence technique. **MITRE ATT&CK:** T1053.005 (Scheduled Task/Job: Scheduled Task)

title: Scheduled Task Creation

logsource: {product: windows, service: security}

detection:

selection:

EventID: 4698

condition: selection

level: high

Detection Logic: Event ID 4698 (Security log) or Sysmon Event ID 1 where `Image` = `schtasks.exe` and `CommandLine` contains `/create`. **Severity:** High. **False Positives:** Software installers legitimately register update tasks — check `TaskContent` for the actual command being scheduled. **Investigation Steps:** Extract the task's action (what binary/script it runs, and its trigger). A task running from `%TEMP%` or `%APPDATA%` on a timer is a strong persistence indicator. **Recommended Response:** Remove the task (`schtasks /delete`), preserve a copy of its XML definition as evidence first.

10.10 Service Creation

Description: Detects new Windows service installation — another top persistence vector. **MITRE ATT&CK:** T1543.003 (Create or Modify System Process: Windows Service)

title: New Service Installed

logsource: {product: windows, service: system}

detection:

selection:

EventID: 7045

condition: selection

level: high

Detection Logic: System Event ID 7045, or Sysmon Event ID 1 for `sc.exe create`.
Severity: High. **False Positives:** Legitimate software installs — check `ImagePath` for unsigned binaries or paths in user-writable directories. **Investigation Steps:** Check `ImagePath`, `ServiceType` (kernel driver = higher concern), and the parent process that created it. **Recommended Response:** Stop and delete the service (`sc delete <name>`), acquire the binary for analysis before removal.

10.11 Mimikatz Indicators

Description: Detects command-line and behavioral indicators consistent with Mimikatz (credential dumping tool) — not signature-based, since binaries are trivially renamed. **MITRE ATT&CK:** T1003.001 (OS Credential Dumping: LSASS Memory)

title: Mimikatz-style Command Line Indicators

logsource: {product: windows, category: process_creation}

detection:

selection:

CommandLine|contains:

- 'sekurlsa::'

- 'lsadump::'

- 'privilege::debug'

condition: selection

level: critical

Detection Logic: Command-line substring match — attackers renaming `mimikatz.exe` doesn't change the module command syntax typed at runtime. **Severity:** Critical. **False Positives:** Near zero — these strings have no legitimate use outside red-team/pentest engagements (which should be pre-notified to the SOC). **Investigation Steps:** Immediately pull the process tree, check `ParentImage`, and correlate with Event ID 10 (LSASS access, 10.12) which almost always accompanies this. **Recommended Response:** Immediate host isolation, force credential reset for any account active on the host at time of execution, treat as active incident.

10.12 LSASS Access

Description: Detects processes opening a handle to `lsass.exe` — the actual mechanism behind credential dumping, tool-agnostic. **MITRE ATT&CK:** T1003.001

title: LSASS Process Access

logsource: {product: windows, category: process_access}

detection:

selection:

TargetImage|endswith: '\lsass.exe'

GrantedAccess:

- '0x1010'

- '0x1410'

- '0x1438'

- '0x143a'

condition: selection

level: critical

Detection Logic: Sysmon Event ID 10, filtered to access rights masks associated with memory reading (not the far more common, benign `0x1000` access from AV/EDR itself — exclude known EDR `SourceImage` paths). **Severity:** Critical. **False Positives:** Legitimate AV/EDR engines also access LSASS — must whitelist by known-good `SourceImage` hash/path. **Investigation Steps:** Identify `SourceImage` (the process reading LSASS). If unsigned or from a non-standard path, this is confirmed credential access. **Recommended Response:** Same as 10.11 — isolate, reset credentials, treat as confirmed incident, not a suspected one.

10.13 RDP Login

Description: Detects RDP (RemoteInteractive) logons, particularly from unexpected sources — a primary lateral movement vector. **MITRE ATT&CK:** T1021.001 (Remote Services: RDP)

title: RDP Logon

logsource: {product: windows, service: security}

detection:

selection:

EventID: 4624

LogonType: 10

condition: selection

level: medium

Detection Logic: Event ID 4624 with `LogonType: 10`. **Severity:** Medium (High if source is an unexpected internal host — indicates lateral movement, not external access, since 3389 shouldn't be internet-facing in this lab). **False Positives:** Legitimate admin remote sessions. **Investigation Steps:** Check `IpAddress` — is it a host that has no business initiating RDP (e.g., a workstation, not the jump box)? Cross-reference with any prior brute-force alerts (10.5) against the same account. **Recommended Response:** Terminate session, verify with account owner, review what the account did during the session (process creation events in that window).

10.14 DNS Tunneling

Description: Detects C2-over-DNS or data exfiltration via DNS queries — abnormal query volume, length, or entropy. **MITRE ATT&CK:** T1071.004 (Application Layer Protocol: DNS)

title: Possible DNS Tunneling - High Query Volume

logsource: {product: zeek, service: dns}

detection:

selection:

query|re: '^[a-f0-9]{32,}\.'

 # long hex/base32-looking subdomains

timeframe: 1m

condition: selection | count(by=src_ip) > 30

level: high

Detection Logic: Zeek `dns.log` — unusually long, high-entropy subdomain labels, or query volume to a single domain far exceeding normal resolution patterns. **Severity:** High. **False Positives:** Some legitimate CDN/security products use long subdomains (e.g., certificate transparency, telemetry beacons) — baseline your environment's normal DNS traffic first. **Investigation Steps:** Pull the full list of queried subdomains for the suspect domain, decode if they resemble Base32/hex-encoded data chunks. **Recommended Response:** Sinkhole the domain at the firewall/DNS resolver, isolate the querying host.

10.15 Reverse Shell

Description: Detects an outbound connection from a shell/interpreter process to an unusual external port, consistent with a reverse shell callback. **MITRE ATT&CK:** T1059 (Command and Scripting Interpreter), T1071

title: Reverse Shell Indicator

logsource: {product: windows, category: network_connection}

detection:

selection:

Image|endswith:

- '\cmd.exe'
- '\powershell.exe'
- '\bash'
- '\nc.exe'

DestinationPort:

- 4444
- 1234
- 9001

condition: selection

level: critical

Detection Logic: Sysmon Event ID 3 (or auditd `connect` syscall on Linux) where the initiating process is a shell/interpreter and the destination port is a common attacker default (4444 = Metasploit default) or simply not a standard service port. **Severity:** Critical. **False Positives:** Low — legitimate shells rarely initiate outbound connections on their own; tune the port list to your environment, since real attackers use arbitrary ports too (this rule is a starting heuristic, not a complete one). **Investigation Steps:** Confirm the destination IP against MISP/threat intel. Pull the parent process — how did the shell get spawned (phishing macro, web shell, exploit)? **Recommended Response:** Immediate network isolation of host, block destination IP at firewall, begin full incident response cycle (Section 16).

10.16 Web Shell

Description: Detects a web server process (IIS/Apache/nginx worker) spawning a command shell — the classic web shell execution pattern. **MITRE ATT&CK:** T1505.003 (Server Software Component: Web Shell)

title: Web Server Spawning Shell Process

logsource: {product: windows, category: process_creation}

detection:

selection:

ParentImage|endswith:

- '\w3wp.exe'
- '\httpd.exe'
- '\nginx.exe'

Image|endswith:

- '\cmd.exe'
- '\powershell.exe'

condition: selection

level: critical

Detection Logic: Sysmon Event ID 1, **ParentImage** is a web server worker process, **Image** is a shell — a legitimate web application almost never does this. **Severity:** Critical. **False Positives:** Near zero for typical web apps; some legacy CGI-based applications may trigger this legitimately — verify against known application inventory. **Investigation Steps:** Identify the web root, search for recently modified/created files (**.aspx**, **.php**, **.jsp**) matching the alert timestamp — this is the dropped web shell file itself. **Recommended Response:** Quarantine the file, review web server access logs for the upload vector (often an unrestricted file upload vulnerability), patch/isolate the application.

10.17 Malware Execution (generic)

Description: Detects execution of a binary matching a known-malicious hash (via MISP enrichment) or YARA signature match on disk. **MITRE ATT&CK:** T1204 (User Execution) / T1587.001 depending on delivery

title: Known-Malicious Hash Execution

logsource: {product: windows, category: process_creation}

detection:

selection:

Hashes|contains: '%MISP_KNOWN_MALICIOUS_SHA256%' # populated dynamically via MISP integration

condition: selection

level: critical

Detection Logic: This is where the MISP integration ([02-Installation-and-SIEM-Setup.md](#), §10) closes the loop — Sysmon Event ID 1 always logs a SHA256 hash; Wazuh's [custom-misp.py](#) integration checks that hash against ingested abuse.ch data automatically. **Severity:** Critical. **False Positives:** Very low for confirmed hash matches; possible if feed data is stale/incorrectly tagged — always cross-verify with a second source (VirusTotal) before final classification. **Investigation Steps:** Full YARA scan of the host ([yara -r rules/ /path](#)) to check for related dropped files, check Sysmon Event ID 11 (FileCreate) around the same timestamp for the delivery mechanism. **Recommended Response:** Isolate, full IR cycle, submit sample hash back to MISP as a confirmed local sighting if you have a private instance.

10.18 Sophos: Firewall/Admin Authentication Failure

Description: Detects failed login attempts against the Sophos WebAdmin console, User Portal, or CLI — the perimeter equivalent of detection 10.3, but for the device that sits in front of everything else. **MITRE ATT&CK:** T1110 (Brute Force) / T1133 (External Remote Services) — this depends on whether the source is external (T1133 relevant) or internal (insider/lateral)

Detection Logic (Wazuh-native — no Sigma backend exists for Sophos, so given directly as rule XML):

```
<group name="sophos,">
```

```
<rule id="110106" level="8">
```

```
<decoded_as>sophos-firewall</decoded_as>
```

```
<field name="sophos.log_component">Firewall Authentication</field>
```

```
<field name="sophos.status">Failed</field>
```

```
<description>Sophos: authentication failure for $(dstuser) from $(srcip)</description>
```

```
<mitre><id>T1110</id></mitre>
```

```
</rule>
```

```
</group>
```

Correction note: an earlier draft of this rule matched `log_component == "CLI"`. Real Sophos syslog samples (verified against the Sophos Community's own published message-ID examples) show ordinary WebAdmin/User Portal authentication events use `log_component="Firewall Authentication"` — "CLI" is a narrower, separate component for console/SSH admin sessions specifically. Matching only "CLI" would miss the far more common WebAdmin brute-force case this rule is meant to catch. See [08-Sophos-Firewall-Integration.md](#) §4 for the corrected decoder this rule depends on.

Severity: High. **False Positives:** Legitimate users mistyping credentials — low volume, ignore; only escalate on threshold (pair with a frequency rule, same pattern as 10.5). **Investigation Steps:** Pull `srcip` — internal (lateral/insider concern) or external (internet-facing WebAdmin exposure, which itself may be a finding worth flagging regardless of whether the brute force succeeds). Cross-reference MISP for the source IP. **Recommended Response:** If external and volume is high, block at the Sophos rule level itself (the device generating the alert can also enforce the block — note this dual role in your report, it's a real architectural point: the sensor and the enforcement point are the same device here, unlike the endpoint detections).

10.19 Sophos: Perimeter Deny Burst (External Port Scan / Probing)

Description: Detects a burst of firewall "Deny"/"Would deny" events from one external source in a short window — the perimeter equivalent of detection 10.6, but catching scans that never reach an internal host because Sophos already dropped them. **MITRE ATT&CK:** T1595.001 (Active Scanning: Scanning IP Blocks) / T1046 **Detection Logic:**

```
<rule id="110110" level="6" frequency="15" timeframe="60">
```

```
<if_matched_sid>110109</if_matched_sid>
```

```
<description>Sophos: possible external port scan — 15+ denied connections from $(srcip) in 60s</description>
```

```

<mitre><id>T1595.001</id></mitre>

</rule>

<rule id="110109" level="3">

  <decoded_as>sophos-firewall</decoded_as>

  <field name="sophos.log_component">Firewall Rule</field>

  <field name="sophos.status">Deny</field>

  <description>Sophos: firewall rule denied a connection</description>

</rule>

```

Detection Logic explanation: rule 110109 is the low-severity base match (every single deny — too noisy to alert on alone); rule 110110 uses Wazuh's [frequency/timeframe/if_matched_sid](#) mechanism to escalate only when the SAME base rule fires 15+ times in 60 seconds — this is the correct Wazuh pattern for "N occurrences of X in Y seconds" (same mechanism used in 10.5), not a Sigma rule, since this needs Wazuh's own aggregation engine. **Severity:** Medium (High if the source has never been seen before — pair with a "first-seen source IP" hunting query, exercise 6 in Section 13). **False Positives:** Internet background noise (mass scanners like Shodan/Censys constantly probe any public IP) — for a home-internship lab this will fire constantly on the WAN side if Sophos's WAN interface is genuinely internet-facing; scope this rule to your LAN-side zone only, or you'll drown in noise from the public internet, not your lab. **Investigation Steps:** Check which ports were targeted ([sophos.log_component=Firewall Rule](#) entries in the burst) — a scan hitting only 443/80 is very different from one hitting RDP/SMB ports. **Recommended Response:** For internet-sourced noise: no action, this is expected background radiation of being on the internet — document this expectation explicitly in your report so you're not treating normal internet noise as a real incident. For LAN-side sourced bursts: treat as 10.6.

10.20 Sophos: VPN Portal Authentication Brute Force

Description: Detects repeated failed SSL VPN authentication attempts — the externally-reachable service most likely to be directly targeted in a real environment. **MITRE ATT&CK:** T1133 (External Remote Services), T1110.001 **Detection Logic:**

```

<rule id="110103" level="3">

```



```

<decoded_as>sophos-firewall</decoded_as>

<field name="sophos.log_component">VPN</field>

<description>Sophos: VPN connection event, component=$(sophos.log_component), from
$(srcip)</description>

</rule>

<rule id="110111" level="10" frequency="8" timeframe="300">

  <if_matched_sid>110103</if_matched_sid>

  <same_source_ip />

  <description>Sophos: SSL VPN brute force — 8+ attempts from $(srcip) in 5
minutes</description>

  <mitre><id>T1110.001</id></mitre>

</rule>

```

Correction note (this pass): an earlier version of this detection defined only rule 110111 (the frequency escalation) while referencing rule 110103 via `if_matched_sid` — but 110103 itself was never actually defined anywhere in this project. That's a dangling reference: 110111 could never fire, because it was waiting on a base match that didn't exist. Rule 110103 is now defined above, paired with the new `sophos-vpn` decoder in

`08-Sophos-Firewall-Integration.md` §4.3, which deliberately uses a loose "contains VPN" pattern rather than a guessed exact string — see that file for the full reasoning.

Verification required before trusting this rule — flagged explicitly, not glossed over: the exact `log_component` value Sophos uses for SSL VPN authentication events was **not confirmed** against a real captured sample during this project's research (the samples available covered Firewall Rule, Firewall Authentication, and IDP/IPS families, not VPN specifically). Rule 110103 above now has a real definition (paired with the `sophos-vpn` decoder in `08-Sophos-Firewall-Integration.md` §4.3, which was added in the same pass this note was corrected) — but its field match is a deliberately loose "contains VPN" pattern, not a confirmed exact string. Validate with `wazuh-logtest` against a real captured VPN login line (§3.2 of the same file) and tighten the pattern once you know the real value. This is a different, smaller problem than what existed before this pass: previously 110103 didn't exist at all, so 110111 could never fire under any circumstance; now the chain is structurally complete and only the exact field pattern needs confirmation. **Severity:** High. **False Positives:** A user who forgot their VPN password and retries manually — distinguish by checking if the account subsequently succeeds shortly after (normal) vs. never succeeds while the source IP tries multiple different

accounts (spray pattern, more serious). **Investigation Steps:** Same as 10.5/10.13 — check for a subsequent successful VPN connection from the same source immediately after the failure burst (indicates the brute force worked). **Recommended Response:** Disable the targeted account, block the source IP at the Sophos firewall rule level, force a credential reset.

Coverage Matrix (for MITRE ATT&CK Navigator import)

Tactic	Techniques Covered
Reconnaissance	T1046, T1595.001 (perimeter — new)
Execution	T1059.001
Persistence	T1053.005, T1543.003, T1136
Privilege Escalation	T1548
Defense Evasion	T1027
Credential Access	T1003.001
Discovery	T1046
Lateral Movement	T1021.001
Command and Control	T1071.004, T1071
Exfiltration	T1071.004
Initial Access	T1133 (external remote services — new, perimeter)
Impact	(add during Advanced Challenges, Section 21)

Export this table as JSON via the ATT&CK Navigator layer format for a visual coverage heatmap — a strong artifact for both the GitHub repo and interview conversations.

Continue to: [04-Threat-Simulation-and-Investigation.md](#), or
[08-Sophos-Firewall-Integration.md](#) for the perimeter build.